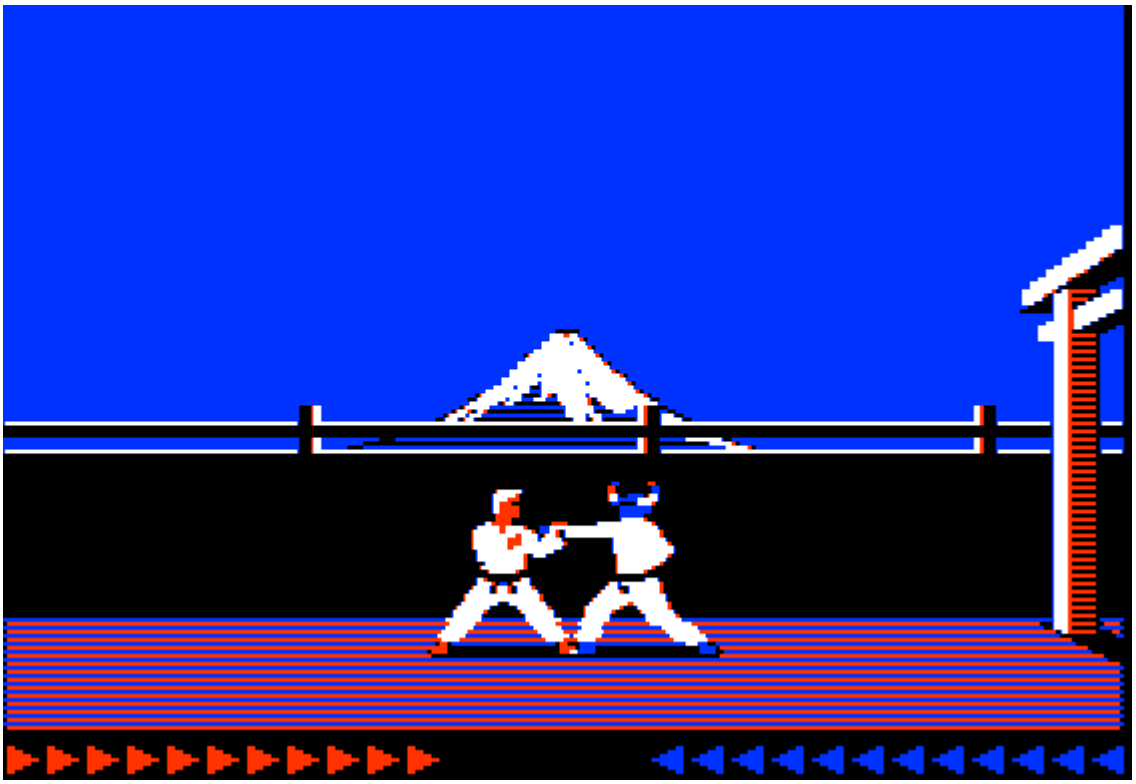
Projeto – Jogo de Luta em Assembly 8086

O projeto consiste no desenvolvimento de um **jogo de luta para dois jogadores**, inspirado em jogos clássicos como *Karateka*, no qual dois lutadores se enfrentam em uma arena. Cada jogador poderá movimentar seu personagem, atacar o adversário e executar movimentos defensivos.

Cada golpe válido, seja **soco ou chute**, reduzirá em uma unidade a energia do adversário. Cada lutador inicia o round com **10 unidades de energia** e, portanto, após receber 10 golpes válidos, perde o round. A partida será disputada em formato **melhor de três rounds**, sendo declarado vencedor o primeiro jogador que conquistar dois rounds.

Cada round terá duração máxima de **3 minutos**. O tempo restante deverá ser apresentado na tela. Caso o tempo termine antes que um dos jogadores perca toda a energia, vencerá o jogador que possuir maior quantidade de energia. Em caso de empate, deverá ser adotado um critério de desempate claramente definido no código.

Antes do início da partida, deverá ser apresentada uma **tela de seleção dos lutadores**, permitindo aos jogadores escolher diferentes cores disponíveis. O Jogador 1 realizará a seleção utilizando as teclas **W/S**, enquanto o Jogador 2 utilizará as **setas para cima/baixo**. A tecla **Enter** dará início à partida após as escolhas.



Durante a luta, os controles serão:

Ação	Jogador 1	Jogador 2
Esquerda	A	←
Direita	D	→
Pular	W	↑
Abaixar	S	↓
Chute	R	N
Soco	T	M

Os movimentos de **pular e abaixar** poderão ser utilizados como mecanismos de defesa, permitindo evitar determinados ataques do adversário.

Requisitos funcionais e de implementação

1. Interface gráfica: A arena deverá possuir delimitadores superior, inferior, esquerdo e direito. A interface deverá apresentar, no mínimo, as barras ou indicadores de energia dos dois jogadores, o tempo restante do round e o número de rounds vencidos por cada jogador.

2. Tratamento do teclado: O teclado deverá ser tratado por meio da **interrupção de hardware INT 9h**, tomando como referência a implementação do programa *Tecbuf*. **Não será permitido utilizar a INT 21h para leitura das teclas durante o jogo.**

3. Ataques e colisões: Socos e chutes somente deverão causar dano quando houver uma condição válida de colisão/proximidade entre os lutadores. Cada ataque válido reduzirá em uma unidade a energia do adversário.

4. Finalização do round e da partida: O round termina quando um jogador receber 10 golpes válidos ou quando o tempo de 3 minutos se esgotar. O primeiro jogador a vencer dois rounds será declarado vencedor da partida.

5. Game Over: Ao final da partida, deverá ser apresentada uma tela com a mensagem **“Game Over”**, identificação do vencedor e um menu perguntando se os jogadores desejam iniciar uma nova partida. A tecla **Y** deverá reiniciar o jogo e **N** encerrar o programa.

6. Encerramento manual: A tecla **Q** poderá ser utilizada a qualquer momento para solicitar o encerramento do jogo. Antes de finalizar, deverá ser apresentado um menu de confirmação **Sim/Não (Y/N)**. Caso o usuário selecione N, o jogo deverá retornar ao estado anterior.

7. Pausa: A tecla **P** deverá alternar entre os estados de execução e pausa. Durante a pausa, o jogo deverá permanecer suspenso por tempo indefinido e apresentar uma indicação visual de **“PAUSED”**. Uma nova pressão da tecla P retomará a execução do ponto em que foi interrompida, sem contabilizar o período de pausa no tempo do round.

8. Temporização: O controle do tempo do jogo deverá ser implementado utilizando mecanismos de temporização compatíveis com o 8086/PC, sem depender de atrasos baseados exclusivamente em laços de software.

9. Restauração do sistema: Ao encerrar o programa, independentemente de a finalização ocorrer pelo Game Over ou pela tecla Q, deverão ser restaurados adequadamente o **modo de vídeo original e o vetor original da interrupção de teclado**, garantindo o retorno seguro ao DOS.

10. Organização do software: O código-fonte deverá ser **claramente comentado e modularizado em arquivos distintos**, separando, sempre que possível, funcionalidades como inicialização, interface gráfica, teclado, temporização, jogadores, colisões e lógica do jogo.

11. Compilação: Todo o projeto deverá ser compilado/montado por meio de um **Makefile**, responsável pelas dependências, montagem dos diferentes módulos, ligação e geração do executável final.

12. Restrição técnica: A lógica principal do jogo deverá ser desenvolvida em **Assembly 8086**, demonstrando a utilização adequada de interrupções, acesso à memória, modularização, manipulação gráfica e interfaceamento com hardware.